DAY 50

Cloud Deployment of Machine Learning Models

60-Day Data Science Learning Journey

Practical guide: prepare a Flask ML API for cloud hosting, configure dependencies and start commands, and test deployed endpoints.

1. What is cloud deployment?

Cloud deployment means running an application on remote infrastructure managed by a cloud provider. Instead of relying on your own laptop, the hosted service runs the application and provides a URL that users can access, subject to its access settings.

Local example: http://127.0.0.1:5000. A cloud platform assigns a public service URL after deployment succeeds.

2. Why deploy an ML model?

• Share predictions through an API.

• Connect a website or mobile application to your model.

• Run the service without keeping your personal computer on.

• Use managed deployment, logging, and infrastructure tools.

• Scale resources when the workload and budget require it.

Cloud hosting does not automatically improve model accuracy. It hosts the application that serves predictions.

3. Common hosting platforms

Render hosts web services and APIs from source repositories. Hugging Face Spaces is useful for ML demos and interactive apps. Railway offers application and database deployment workflows. Vercel is commonly used for frontend apps and supported serverless functions. Check current pricing, runtime support, limits, and service behavior before choosing.

4. Deployment workflow

```
Developer -> GitHub repository -> Cloud build
                        |
                   Install dependencies
                   Train or retrieve model
                   Start Gunicorn + Flask
                        |
                    Public URL
                        |
                 HTTPS API request
                        |
                   Flask -> ML model
                        |
                      JSON response
```

5. Project structure

```
cloud_ml_deployment/
|-- app.py
|-- train_model.py
|-- requirements.txt
|-- render.yaml
|-- .gitignore
|-- README.txt
```

The supplied Python file is a project generator. Run it once to create the project folder and its source files.

6. Train and save the model

train_model.py loads scikit-learn's built-in diabetes regression dataset, splits it into training and testing data, trains a RandomForestRegressor, reports MAE and R2, and saves the model with joblib.

```
python train_model.py
```

Saving the model means the API can load it at startup instead of retraining for every prediction.

7. Understand the Flask API

GET / returns service information. GET /health provides a health-check response. POST /predict accepts a JSON object containing exactly 10 numeric features and returns a prediction.

```
{
```

```
  "features": [0.038, 0.05, 0.06, 0.02, -0.04,
               -0.03, -0.04, -0.003, -0.01, 0.02]
}
```

The API validates the JSON structure, list length, numeric conversion, and finite values before calling the model.

8. Dependencies and reproducibility

requirements.txt lists the packages needed by the application. The starter file uses compatible version ranges. For a production service, test exact versions and pin them to make builds more reproducible.

```
Flask>=3.0,<4.0
gunicorn>=21.2,<24.0
joblib>=1.3,<2.0
numpy>=1.26,<3.0
scikit-learn>=1.4,<2.0
```

9. Why Gunicorn?

Flask's built-in server is intended for local development. Gunicorn is a production-oriented WSGI server used to run Flask applications in hosted environments.

```
gunicorn app:app
```

The first app refers to the Python module app.py. The second app refers to the Flask application object named app inside that module.

10. Local testing

```
python -m pip install -r requirements.txt
python train_model.py
python app.py
```

Open http://127.0.0.1:5000/ and http://127.0.0.1:5000/health. Test POST /predict with the sample JSON shown above.

11. Render deployment

Push the generated project folder to a GitHub repository.

Create a Render Web Service and connect the repository.

If the project is inside a subfolder, set the service root directory to that folder.

Set build command: pip install -r requirements.txt && python train_model.py.

Set start command: gunicorn app:app.

Deploy and wait for the build and startup process to complete.

Open the real service URL assigned by Render and test /, /health, and /predict.

The project includes render.yaml with equivalent service settings. Never invent a public URL. Use the actual URL provided after deployment.

12. Environment variables and secrets

Use environment variables for configuration that changes between environments. Store API keys, passwords, and tokens in the provider's secure environment or secret settings. Do not commit credentials or .env files to GitHub.

13. Logs and troubleshooting

Build fails: Check build logs, root directory, requirements.txt, Python compatibility, and dependency versions.

App fails to start: Check the start command and confirm that app.py contains a Flask object named app.

Model file missing: Confirm that the build command trains the model or that the artifact is retrieved securely.

Health check fails: Confirm /health returns a successful response.

First request is slow: Some free services may sleep when idle. Check current provider behavior.

14. Production checklist

• Use HTTPS and authentication for protected endpoints.

• Validate inputs and limit request sizes.

• Keep credentials in secure settings.

• Pin tested dependencies and update them regularly.

• Monitor latency, errors, and resource usage.

• Add rate limiting and abuse protection.

• Use a production server such as Gunicorn.

• Understand usage limits, backups, and billing.

15. Practical tasks and solutions

Task 1: Generate the folder. **Solution:** Run python Day-50-Cloud-ML-Deployment-Project-Generator.py.

Task 2: Train the model. **Solution:** Run python train_model.py and check for diabetes_model.joblib.

Task 3: Start the API. **Solution:** Run python app.py and open http://127.0.0.1:5000/.

Task 4: Explain Gunicorn. **Solution:** It runs the Flask app object in a hosted WSGI environment.

Task 5: Explain why secrets should not be committed. **Solution:** Exposed credentials can allow unauthorized access to services or data.

16. Knowledge check and answers

1. What is cloud deployment?

2. What does requirements.txt contain?

3. Why load a saved model rather than retrain for each request?

4. What does gunicorn app:app mean?

5. What is a health-check endpoint used for?

6. Where should API keys be stored?

7. Does free hosting guarantee uninterrupted service?

Answers: 1. Running an app on remote cloud infrastructure. 2. Python dependencies. 3. To avoid repeated training work on each prediction. 4. Run the Flask object named app from module app. 5. Check whether the service is healthy. 6. Secure environment or secret settings. 7. No, free plans have limits and may sleep or change.

17. Dataset limitation

The scikit-learn diabetes regression dataset is used only to demonstrate deployment. Its output is not a diagnosis, personal medical risk assessment, or treatment recommendation. Do not use it for clinical decisions.

18. References

Render: https://render.com/docs

Flask: https://flask.palletsprojects.com/

Gunicorn: https://docs.gunicorn.org/

scikit-learn: https://scikit-learn.org/stable/

GitHub: https://docs.github.com/